Pivot Tables

To create pivot tables in Python and Pandas using the .pivot_table() method. Being able to quickly summarize data is an important skill to be able to get a sense of what your data

looks like. The function is quite similar to the .groupby() method also available in Pandas,

How to Build a Pivot Table in Python

A pivot table is a table of statistics that helps summarize the data of a larger table by “pivoting” that data. Microsoft Excel popularized the pivot table, where they’re known as PivotTables. Pandas gives access to creating pivot tables in Python using the .pivot_table() function.

```
# The syntax of the .pivot_table() function

import pandas as pd

pd.pivot_table(

    data=,

    values=None,

    index=None,

    columns=None,

    aggfunc='mean',

    fill_value=None,

    margins=False,

    dropna=True,
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
    margins_name='All',
    observed=False,
    sort=True
)
```

The method takes a DataFrame and then also returns a DataFrame.

Parameter	Default Value	Description
data =		The DataFrame to pivot
values=		The column to aggregate (if blank, will aggregate all numerical values)
index=		The column or columns to group data by. A single column can be a string, while multiple columns should be a list of strings
columns=		The column or columns to group data by. A single column can be a string, while multiple columns should be a list of strings
aggfunc=	'mean'	A function or list of functions to aggregate data by
fill_value=		Value to replace missing values with
margins=	False	Add a row and column for totals
dropna=	True	To choose to not include columns where all entries are NaN
margins_name=	'All'	Name of total row/column
observed=	False	Only for categorical data – if True will only show observed values for categorical groups

smartechnologies
UDIFY THE FUTURE WITH AI
4th Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191210

sort= True Whether to sort the resulting values

Loading a Sample Pandas DataFrame

To follow along with this tutorial, let's load a sample Pandas DataFrame. We can load the DataFrame from the file hosted on my GitHub page, using the pd.read_excel() function. Then we can print out the first five records of the dataset using the .head() method.

```
# Loading a Sample Pandas DataFrame
import pandas as pd
df = pd.read_excel('https://github.com/datagy/mediumdata/ \
       raw/master/sample_pivot.xlsx', parse_dates=['Date'])
# dataset can be directly pulled from github/cloud so it
#doesn't required to be downloaded
print(df.head())
```

Output:

```
Date Region                 Type  Units  Sales
0 2020-07-11   East  Children's Clothing   18.0    306
1 2020-09-23  North  Children's Clothing   14.0    448
2 2020-04-02  South    Women's Clothing   17.0    425
3 2020-02-28   East  Children's Clothing   26.0    832
4 2020-03-19   West    Women's Clothing    3.0     33
```

smectechnologies GUIDING THE FUTURE WITH TECH 2017 II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala Ph. No: +91 97781917 05

Column Name	Description
Date	Date of transaction
Region	The region of the transaction
Type	The type of clothing sold
Units	The number of units sold
Sales	The cost of the sale

Creating a Pivot Table in Pandas

Let's create your first Pandas pivot table. At a minimum, we have to pass in some form of a group key, either using the index= or columns= parameters. In the examples below, we're using the Pandas function, rather than the DataFrame function. Because of this,

we need to pass in the data= argument. If we applied the method to the DataFrame directly, this would be implied.

Aggregating Only Certain Columns in a Pandas Pivot Table

Pandas allows us to pass in either a single string representing one column or a list of strings representing multiple columns. Let's now modify our code to only calculate the mean for a single column,

```
# Loading a Sample Pandas DataFrame
import pandas as pd
df=pd.read_excel('https://github.com/datagy/mediumdata/raw/master/ \
                  sample_pivot.xlsx', parse_dates=['Date'])
# Creating your first Pandas pivot table
```

```
pivot = pd.pivot_table(\
    data=df,\
    index='Region',\
    values=['Sales','Units'] #values='Sales' # for single column \
)
print(pivot)
```

Output:

	Sales	Units
Region		
East	*408.182482*	*19.732360*
North	*438.924051*	*19.202643*
South	*432.956204*	*20.423358*
West	*452.029412*	*19.294118*

Let's break down what happened here:

We created a new DataFrame called sales_by_region, which was created using the pd.pivot_table() function We passed in our DataFrame, df, and set the index='region', meaning data would be grouped by the region column, values=['Sales',Units'] means select the numeric column required for the grouping.

Working with Aggregation Methods in a Pandas PivotTable

Now that you've created your first pivot tablein Pandas, let's work on changing the aggregation methods. This allows you to specify how you want your data aggregated. This is where the power of Pandas really comes through, allowing you to calculate complex analyses with ease.

smctechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Palarivattom, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 97 8810 9015

Specifying Aggregation Method in a Pandas Pivot Table

You can use the aggfunc= (aggregation function) parameter to change how data are

aggregated in a pivot table. By default, Pandas will use the .mean() method to aggregate data. You can pass a named function, such as 'mean', 'sum', or 'max', or a function callable such as np.mean.

Let's now try to change our behavior to produce the sum of our sales across all regions:

```
 # Loading a Sample Pandas DataFrame
import pandas as pd
df=pd.read_excel('https://github.com/datagy/mediumdata/ \
   raw/master/sample_pivot.xlsx', parse_dates=['Date'])
# Specifying the Aggregation Function
pivot = pd.pivot_table(\
    data=df,\
    index='Region',\
    values='Sales',\
    aggfunc='sum'\
)
print(pivot)
```

Output:

```
         Sales

Region

East    167763
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

North 138700

South 59315

West 61476

Multiple Aggregation Method in a Pandas DataFrame

Similarly, we can specify multiple aggregation methods to a Pandas pivot table. This is quite easy and only requires you to pass in a list of functions and the function will be applied to all values columns. Let's produce aggregations for both the mean and the sum:

```
# Loading a Sample Pandas DataFrame
import pandas as pd
df=pd.read_excel('https://github.com/datagy/mediumdata/ \
        raw/master/sample_pivot.xlsx', parse_dates=['Date'])
pivot = pd.pivot_table(\
    data=df,\
    index='Region',\
    values='Sales',\
    aggfunc=['mean', 'sum']\
)
print(pivot)
```

Output:

	mean	sum
	Sales	Sales
Region		

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
East    408.182482  167763
North   438.924051  138700
South   432.956204   59315
West    452.029412   61476
```

Specifying Different Aggregations Per Column

Now, imagine you wanted to calculate different aggregations per column. In order to do this, you can pass in a dictionary containing the following key-value pair format: 'column': function. Let's say we wanted to calculate the sum of units and the average number of sales:

```
# Loading a Sample Pandas DataFrame
import pandas as pd
df=pd.read_excel('https://github.com/datagy/mediumdata/ \
     raw/master/sample_pivot.xlsx', parse_dates=['Date'])
pivot = pd.pivot_table(\
    data=df,\
    index='Region',\
    aggfunc={'Sales': 'mean', 'Units': 'sum'}\
)
print(pivot)
```

Output:

```
              Sales   Units
Region
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
East     408.182482  8110.0
North    438.924051  4359.0
South    432.956204  2798.0
West     452.029412  2624.0
```

Custom Aggregations in Pandas Pivot Tables

Pandas also allows us to pass in a custom function into the .pivot_table() function. This greatly extends our ability to work with analyses specifically tailored to your needs! Let's see how we can pass in a function that calculates the mean of a column without any outliers. Pandas comes with a method, .quantiles(), that lets us define a range of values we want to select based on their percentage in the range of data. Let's say we wanted to calculate the average of a column, removing the top and bottom 10% of the data.

```
# Loading a Sample Pandas DataFrame
import pandas as pd
df=pd.read_excel('https://github.com/datagy/mediumdata/ \
       raw/master/sample_pivot.xlsx', parse_dates=['Date'])
# Defining a custom function
import numpy as np
def mean_no_outliers(values):
    no_outliers = values.quantile([0.1, 0.9])
    mean = np.mean(no_outliers)
    return mean
# Specifying custom functions in a Pandas pivot table
```

```
pivot = pd.pivot_table(\
    data=df,\
    index='Region',\
    aggfunc=['mean', mean_no_outliers],\
    values='Sales'\
)
print(pivot)
```

Output:

	mean	mean_no_outliers
	Sales	Sales
Region		
East	408.182482	436.0
North	438.924051	484.5
South	432.956204	434.1
West	452.029412	497.0

This function accepts a single parameter, values, which will be the values passed in by the .pivot_table() function. The values are then filtered using the .quantile() method. Finally, the mean of these values is calculated. Let's see how we can use this (and the normal mean aggregation) in our pivot table, applied to our Sales column.

More Complex Pandas Pivot Tables

smectechnologies
GUIDING THE FUTURE WITH AI
Kaloor, Kochi, Ernakulam, Kerala 682017

Now that you have an understanding of how the .pivot_table() function works in Pandas, let's take a look at how we can expand our understanding.

Add Columns to a Pandas Pivot Table

When we add columns to a Pandas pivot table, we add another dimension to the data. While the index= parameter splits the data vertically, the columns= parameter groups and splits the data horizontally. This allows us to create an easy-to-read table. Let's see how we can use the columns= parameter to split the data by the Type column.

```
# Loading a Sample Pandas DataFrame
import pandas as pd
df=pd.read_excel('https://github.com/datagy/mediumdata/ \
raw/master/sample_pivot.xlsx', parse_dates=['Date'])
# Adding Columns to Our Pandas Pivot Table
pivot = pd.pivot_table(\
    data=df,\
    index='Region',\
    columns='Type',\
    values='Sales'\
)
print(pivot)
```

Output:

```
Type    Children's Clothing  Men's Clothing  Women's Clothing
Region
East             405.743363      423.647541        399.028409
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

North *438.894118* *449.157303* *432.528169*

South *412.666667* *475.435897* *418.924528*

West *480.523810* *465.292683* *419.188679*

Filter data by quarters (4 quarters in a month)

While columns added a horizontal dimension, we can also specify multiple indices when there is a logical hierarchy in our data. For example, we can add a date dimension to our pivot table. Let's use Pandas' built-in date accessors to group our data by quarters. This allows us, then, to visualize our data over a period of time.

```
import pandas as pd
df=pd.read_excel('https://github.com/datagy/mediumdata/ \
      raw/master/sample_pivot.xlsx', parse_dates=['Date'])
pivot = pd.pivot_table(\
    data=df,\
    index=['Region',df['Date'].dt.quarter],\
    columns='Type',\
    values='Sales'\
)
print(pivot)
```

Output:

```
Type           Children's Clothing  Men's Clothing  Women's Clothing
Region Date
East   1                423.241379      369.250000        428.948718
```

smetechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191225

	2	274.800000	445.425000	456.816327
	3	425.382353	506.421053	342.386364
	4	453.866667	405.666667	364.795455
North	1	394.727273	450.869565	489.944444
	2	461.791667	478.916667	406.263158
	3	443.806452	482.947368	432.850000
	4	427.526316	388.478261	393.892857
South	1	509.400000	433.214286	465.785714
	2	376.363636	482.875000	463.687500
	3	430.909091	460.875000	370.937500
	4	353.538462	547.444444	332.571429
West	1	484.266667	532.307692	401.333333
	2	523.444444	394.571429	308.090909
	3	316.857143	406.285714	542.083333
	4	544.454545	467.928571	417.055556

Accessing data in a multi-index Pandas DataFrame_works quite similarly to accessing data in any other DataFrame. However, since now we have two index columns, rather than 1, we can pass in a tuple of indices. Let's say we wanted to access only the intersection of East Region, Quarter 1, and Men's clothing.

```
import pandas as pd
df=pd.read_excel('https://github.com/datagy/mediumdata/ \
     raw/master/sample_pivot.xlsx', parse_dates=['Date'])
pivot = pd.pivot_table(\
    data=df,\
    index=['Region',df['Date'].dt.quarter],\
    columns='Type',\
    values='Sales'\
```

smartechnologies

```
)
print(pivot.loc[('East', 1), "Men's Clothing"])
```

Output:

369.25

Customizing Pandas Pivot Tables

To customize your Pandas pivot tables. This allows you to add even further customizations, such as adding totals and working with missing data. You'll also learn how to fill missing data in a resulting pivot table with a specific value.

Adding Totals to Pandas Pivot Tables

Let's start off by learning how to add totals to a Pandas pivot table. This is controlled by the margins= parameter, which accepts a boolean value. By default, this is set to False, but toggling it to True adds the totals to rows and columns.

```
# Loading a Sample Pandas DataFrame
import pandas as pd
df=pd.read_excel('https://github.com/datagy/mediumdata/ \
      raw/master/sample_pivot.xlsx', parse_dates=['Date'])
# Adding totals to rows and columns
pivot = pd.pivot_table(\
    data=df,\
    index='Region',\
    columns='Type',\
    values='Sales',\
```

```
    margins=True\
)
print('Original data')
print(pivot)

# Renaming column as total in a Pandas pivot table
pivot = pd.pivot_table(
    data=df,
    index='Region',
    columns='Type',
    values='Sales',
    margins=True,
    margins_name='Total'
)
print('Column renamed to Total')
print(pivot)
```

Output:

Type	Children's Clothing	Men's Clothing	Women's Clothing	All
Region				
East	405.743363	423.647541	399.028409	408.182482
North	438.894118	449.157303	432.528169	438.924051
South	412.666667	475.435897	418.924528	432.956204
West	480.523810	465.292683	419.188679	452.029412
All	427.743860	444.257732	415.254717	427.254000

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778601215

Type	Children's Clothing	Men's Clothing	Women's Clothing	Total
Region				
East	405.743363	423.647541	399.028409	408.182482
North	438.894118	449.157303	432.528169	438.924051
South	412.666667	475.435897	418.924528	432.956204
West	480.523810	465.292683	419.188679	452.029412
Total	427.743860	444.257732	415.254717	427.254000

By default, Pandas will name the totals 'All'. If you wanted to rename these labels, you can use the margins_name= parameter to pass in a string to relabel the values.

Dealing with Missing Data in a Pandas Pivot Table

When Pandas encounters a cross-section where no data exists, it'll include a NaN value in the resulting pivot table. Let's modify our DataFrame to include some missing data and calculate a pivot table

```
# Loading a Sample Pandas DataFrame
import pandas as pd
df=pd.read_excel('https://github.com/datagy/mediumdata/ \
       raw/master/sample_pivot.xlsx', parse_dates=['Date'])
# Adding and seeing missing data in a Pandas pivot table
import numpy as np
```

```
df.loc[(df['Region'] == 'East') & (df['Type'] == "Children's
Clothing"), 'Sales'] = np.NaN
pivot = pd.pivot_table(
    data=df,
    index='Region',
    columns='Type',
    values='Sales',
)
print(pivot)
```

Output:

Type	Children's Clothing	Men's Clothing	Women's Clothing
Region			
East	NaN	423.647541	399.028409
North	438.894118	449.157303	432.528169
South	412.666667	475.435897	418.924528
West	480.523810	465.292683	419.188679

It may not always be ideal to see a NaN value, especially for non-technical audiences. Because of this, Pandas provides a parameter, fill_value=, which enables you to pass in a value to fill these missing data points.

```
# Loading a Sample Pandas DataFrame
import pandas as pd
df=pd.read_excel('https://github.com/datagy/mediumdata/ \
```

smetechnologies
GUIDING THE FUTURE WITH AI
II floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Phone: +91 9778191215

```
      raw/master/sample_pivot.xlsx', parse_dates=['Date'])
# Filling Missing Values in a Pandas Pivot Table
import numpy as np
df.loc[(df['Region'] == 'East') & (df['Type'] == "Children's
Clothing"), 'Sales'] = np.NaN

pivot = pd.pivot_table(
    data=df,
    index='Region',
    columns='Type',
    values='Sales',
    fill_value=0
)
print(pivot)
```

Output:

Type	Children's Clothing	Men's Clothing	Women's Clothing
Region			
East	0.000000	423.647541	399.028409
North	438.894118	449.157303	432.528169
South	412.666667	475.435897	418.924528
West	480.523810	465.292683	419.188679

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 7778191275

Sorting Pandas Pivot Table Data

First generate the DataFrame and then pass in a method to sort the data. Now, you can simply pass in the sort=True argument to help sort your resulting DataFrame.

```
# Loading a Sample Pandas DataFrame
import pandas as pd
df=pd.read_excel('https://github.com/datagy/mediumdata/ \
      raw/master/sample_pivot.xlsx', parse_dates=['Date'])
# Sorting a Pandas Pivot Table
pivot = pd.pivot_table(\
    data=df,\
    index='Region',\
    values='Sales',\
    sort=True \
)
print(pivot)
```

Output:

```
              Sales
Region
East     408.182482
North    438.924051
South    432.956204
West     452.029412
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

By default, Pandas will sort the pivot table in ascending order. Unfortunately, for more complex sorting (such as across different columns), you would still need to chain the .sort_values() method.

Filtering Python Pivot Tables

In this section, you'll learn how to filter a Pandas pivot table. Because pivot tables can often be quite large, filtering a pivot table can focus the results quite a bit. Because the function returns a DataFrame, you can simply filter the DataFrame as you would any other. Let's recreate our pivot table adding up values over quarters and regions.

```
# Loading a Sample Pandas DataFrame
import pandas as pd
df=pd.read_excel('https://github.com/datagy/mediumdata/ \
     raw/master/sample_pivot.xlsx', parse_dates=['Date'])
# Generating a long pivot table
pivot = pd.pivot_table(\
    data=df,\
    index=['Region', df['Date'].dt.quarter],\
    values='Sales'\
)
print(pivot[pivot['Sales'] > pivot['Sales'].mean()])
```

Output:

```
                    Sales

Region Date

North  1     462.142857
```

smectechnologies
GUIDING THE FUTURE WITH AI
1st Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
P.I. No: +91 9778 [illegible] 215

```
         2      442.034884
         3      447.200000
South    1      465.263158
         2      440.628571
West     1      475.000000
         3      444.884615
         4      466.209302
```

smetechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215